Sync Car

1. Objetivo

Implementar em C a sincronização entre vagões e passageiros com pthreads. A solução reserva vagas sem exceder a capacidade, permite retornos simultâneos de station_wait_for_car e mantém o vagão bloqueado até todas as confirmações. A entrega contém código, Makefile, README, relatório e evidências reais de execução.

2. Checklist de requisitos

Requisito	Status	Evidência
C, pthreads e sistemas POSIX	OK	Compilado no Ubuntu/WSL2 com GCC 15.2.0.
Quatro funções da estação	OK	sync_car.c e testes finais.
Um mutex por estação	OK	A struct station possui somente lock.
Sem semáforos ou espera ativa	OK	sync_car.c usa pthread_cond_wait.
Um vagão por estação	OK	Hipótese documentada e exercitada pelos testes.
Embarques simultâneos	OK	Casos com 6, 12 e 32 passageiros.
Makefile gera sync_car e limpa binários	OK	make e make clean registrados no log.
README, PDF, log e código no TAR	OK	Conteúdo conferido no pacote hmrn.tar.
Nomes hmrn/, hmrn.tar e hmrn.pdf	OK	Padrão do login aplicado.
Link do GitHub com README e PDF	Pendente	Repositório remoto ainda não informado.

3. Como reproduzir

Ambiente validado: Ubuntu no WSL2, Linux 6.18.54.1-microsoft-standard-WSL2 x86_64, GCC 15.2.0 e GNU Make 4.4.1.

```
make clean
make test-inicial
make test-integracao
make
./sync_car
make clean
```

Os comandos acima foram executados na versão final. Todos terminaram com código 0. O runner mostra um aviso de parâmetro não utilizado no arquivo recebido; sync_car.c e os testes próprios compilam com -Werror.

4. Arquitetura resumida

Arquivo	Responsabilidade
sync_car.h	Estado da estação e protótipos.
sync_car.c	Inicialização, reserva, espera e confirmação.
sync_car-runner.c	Runner fornecido no enunciado. Inicialização
teste_etapa1.c	dos recursos pthread. Integração,
testes.c	simultaneidade e casos limite. Compilação,
Makefile	testes e limpeza.
evidencias.log	Comandos, horários e saídas reais.

A estação funciona como um monitor. Um mutex protege waiting, free_seats e boarding. car_available acorda passageiros quando chegam vagas; boarding_done acorda o vagão depois das confirmações. A reserva reduz a vaga e aumenta boarding na mesma seção crítica, impedindo duas reservas do mesmo assento.

5. Estratégias e diário de desenvolvimento

5.1 Estratégias

Estratégia	Contexto	Resultado
S1 - monitor por estação	Um mutex, duas condições e contadores separados.	Mantida na versão final.
S2 - signal para passageiros	Variante criada para tentar corrigir um erro	Bloqueou com vagas livres; descartada.

5.2 Diário de tentativas

Quando	Tentativa	Resultado / evidência
31/10/2026 19:29-20:26	Estado inicial e quatro cenários.	execuções concluídas.
01/11/2026 12:03-12:39	Espera bloqueante.	execuções concluídas.
02/11/2026 12:33-13:14	Reserva de assentos.	execuções concluídas.
03/11/2026 22:29-23:19 03-	Confirmação de embarque.	execuções concluídas.
04/11/2026 23:26-00:16	Casos limite.	execuções concluídas.
04/11/2026 14:00-19:12	Variante S2 com signal.	bloqueios controles.
04/11/2026 21:18-21:19	Versão final, integração e runner.	Código 0 e Looks good!.

5.3 Dificuldades

Oestado precisoudistinguir vaga reservada de embarque confirmado; por isso boarding é separado de free_seats. A variante com pthread_cond_signal acordava só um passageiro quando havia várias vagas e deixou o vagão bloqueado. A versão final usa pthread_cond_broadcast para disponibilizar as vagas e mantém pthread_cond_signal apenas para avisar o único vagão. A struct ficou em sync_car.h porque o runner precisa conhecer seu tamanho.

6. Erros e evidências

Prints

```
+ 2026-11-04T14:14:41.112727-03:00 .../lote_t2_01_a_10/teste2
CENARIO=2 PESSOAS=2 VAGAS=2 AUTORIZADOS=1 ESPERANDO=1
```

Figura 1. A variante S2 deixou uma vaga livre e um passageiro esperando.

```
Validação da versão final
+ 2026-11-04T21:18:08-03:00 timeout 35s make test-integracao
PASS: passageiros=1, primeiro_vagao=1, confirmacoes controladas
PASS: passageiros=6, primeiro_vagao=2, confirmacoes controladas
PASS: passageiros=12, primeiro_vagao=12, confirmacoes controladas
PASS: passageiros=32, primeiro_vagao=7, confirmacoes controladas
TESTES DE INTEGRACAO APROVADOS
+ 2026-11-04T21:18:08-03:00 timeout 35s ./sync_car
Looks good!
EXIT=0
```

Figura 2. A integração e o runner da versão final terminaram com sucesso.

6.3 Observação sobre o runner

O runner recebido usa volatile e espera ativa em seu próprio código. Essas escolhas pertencem ao teste fornecido. A implementação em sync_car.c bloqueia com variáveis de condição e não faz polling.

7. Uso de IA

Usei a IA para me ajudar com a setar as configurações do vscode etc pois tinha formatado o pc, usei tambem para me ajudar resolver erros ao longo da construcção: “estou preso me de uma dica de o que esta de errado “. e para verificar se todos os arquivos pedidos estavam na pasta certa, uma skill fez ela alucinar e criou umas coipais formatadas dos arquivos mas nada que tenha me atrapalhado.

8. Reflexão final

Separar reserva e confirmação evitou que o vagão partisse antes do fim do embarque. Um único mutex tornou os predicados mais fáceis de revisar. O experimento com signal mostrou que uma notificação única não atende várias vagas disponíveis. Os testes repetidos ajudaram a observar bloqueios que uma execução isolada poderia esconder. Com mais tempo, eu testaria o código com ThreadSanitizer e em outro sistema POSIX. Também evitaria misturar a limitação do runner com a avaliação da implementação.

9. Checklist final

Item	Confirmado
Compilei do zero seguindo o relatório	Sim.
Rodei os testes e gerei evidencias.log	Sim, Figuras 1 e 2.
Tenho os prints obrigatórios	Sim, zero e contagem negativa.
Testei caso limite e caso inválido	Sim
Preenchi a seção de uso de IA Revisei e removi frases genéricas	. Sim.

10. Se eu tivesse mais duas horas

Executaria ThreadSanitizer em uma suíte que não dependa do volatile do runner. Repetiria a versão final com passageiros chegando durante o embarque. Testaria a compilação no macOS e em outro Linux. Acrescentaria verificações para falhas de inicialização dos recursos pthread. Mediria cobertura dos predicados de espera. Por fim, publicaria o repositório e confirmaria o link do PDF no GitHub.

Materiais consultados

Enunciado Sincronização do Metrô, Guia de Relatórios, sync_car-runner.c fornecido.